



LEKTION 12

DESIGN PATTERN

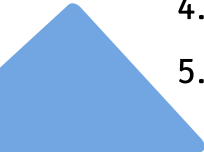
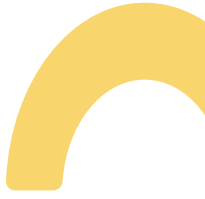
MÅL

EFTER ENDT UNDERVISNING KAN DEN STUDERENDE:

- Redegør for de tre kategorier af DP
- Udvælge en kategori til løsning af problemer



AGENDA

- 
1. Recap
 2. Design Pattern
 3. Structural
 4. Behavioural
 5. Constructional
- 
- 

DESIGN PATTERN

GoF

- Gendkendte mønstre
- Nedskrev/formaliserede dem.
- Opdagede - skabte ikke

Kategorisering af design patterns.

- Structural
Beskriver struktur i software
- Behavioural
Beskriver adfærd i software
- Creational
Har kontrol over levetid af objekter

Hvornår bruges?

- Refaktor - for det meste.
- Under udvikling - når det er indlysende.
- Til at løse problemer

Hvordan skal de bruges?

- Som et redskab
- Forbedre kvalitet
- Forbedre stabilitet

DESIGN PATTERN

HVAD ER DET IKKE!!

1. Hvis man ikke har et problem der skal løses - de skal ikke bruges bare for at bruges.
2. Koden bliver for kompleks - dårlig maintainability og readability
3. Forkert brug kan skabe kobling - bryder mod OOP
4. Når pattern bliver et mål - det er et værktøj til at løse et problem
5. Gør det svære at teste - Vi risikere at de fejl der er, er mere komplicerede
6. Hvis sproget allerede løse det - jeg ved godt C# løser masser af det her :D
7. Der er slet ikke behov for fleksibilitet.

STRUCTURAL

MØNSTRE SOM BESKRIVER/IMPLEMENTERE STRUKTUR TIL KODEN. VI KAN SE DISSE TIDLIG I UDVIKLINGSFASE - TYPISK.

ADAPTER

Tilpasser interface til at forbinde mellem "ikke" kompatibel interfaces.

Skal bruges når dele af koden ikke er kompatibel, er der ønskes en tilpasning. Kan ikke ændre implementation, hvor mange steder skal der så tilpasse?

FACADE

Udstiller interface til at forbinde mellem kompatibel interfaces.

Skal bruges når der skal udstilles et interface til kode. Adapter kobles typisk på for at gøre implementation komplet.

CASE 1

EKSTERNT BIBLIOTEK

Dit system bruger et internt betalingsinterface. Virksomheden køber et eksternt betalingssystem. De to interfaces passer ikke sammen.

PROBLEMET

- Du må ikke ændre eksternt system
- Du vil ikke ændre hele dit system
- Du skal få dem til at tale sammen

BEHOV

Vi ønsker:

- At oversætte mellem to interfaces
- Uden at ændre eksisterende kode
- Uden at sprede tilpasningskode overalt

CASE 2

LOGIN-PROCES

Login kræver:

- Validering
- Hashing
- Databaseopslag
- Token Generering
- Logning

Frontend skal bare kalde:

```
Login(username, password)
```

PROBLEMET

Hvis frontend selv kalder alle delene:

- Komplexitet spredes
- Høj kobling
- Svært at ændre backend

BEHOV

Vi ønsker:

- Én simpel indgang
- Intern kompleksitet skjules
- Tydeligt interface

BEHAVIOURAL

MØNSTRE DER BESKRIVER/IMPLEMENTERE ADFÆRD FOR SYSTEMET.

Kan ses sent i udvikling, da det ikke altid er indlysende det er den adfærd vi ønsker af systemet.

OBSERVER

Observer hændelse der sker. Gør det muligt at subscribe til forskellige hændelse og blive notificeret ved ændring. Kan bruges ved logføring, hver gang der sker en ændring skal det logføres.

COMMAND

Enkapsulere en forespørgelse for at kunne parametisere, sætte i kø eller fortryde handling. Bruges til steder hvor vi ønsker større kontrol over kommando. Dette kunne være database skrivning.

CASE 3

ORDRESTATUS ÆNDRES

Når en ordre ændrer status:

- UI skal opdateres
- Log skal skrives
- Kunde skal have mail
- Lager skal opdateres

PROBLEMET

Hvis Ordre selv kalder alle disse ting:

- Den kender for meget
- Nye funktioner kræver ændring i ordre
- Svært at teste

BEHOV

Vi ønsker:

- Ordre skal kun sige: "Noget er ændret"
- Andre dele kan lytte
- Let at tilføje nye reaktioner

CASE 4

KONTROLLERBAR HANDLING

Et system giver mulighed for:

- Opret
- Slet
- Rediger

Brugeren skal kunne:

- Fortryde
- Gentage
- Logge handlinger
- Sætte handlinger i kø

PROBLEMET

Hvis vi bare kalder metoder direkte:

- Ingen historik
- Ingen fortrydelse
- Ingen fleksibel styring

BEHOV

Vi ønsker:

- At pakke handlinger ind som objekter
- Kunne gemme dem
- Kunne afvikle dem senere
- Kunne fortryde dem

CREATIONAL

MØNSTRE DER BESKRIVER OPRETTELSE AF OBJEKTER, HERUNDER OGSÅ EJERSKAB. DISSE KAN SES FORSKELLIGE STEDER I UDVIKLING, LIGE FRA KONCEPTUELLE DESIGN TIL REFAKTOR.

SINGLETON

Sikre et objekt ad gangen, derved beskytter vi en resourceHolder styr på oprettelse af objekter, med total ejerskab fra at blive brugt af flere på en gang. Eksempler på denne over objektet. Bruges hvis der ønskes høj grad af kunne være database, for at undgå at flere prøver at decoupling og fleksibilitet ved brug af polymorfisme. skrive/læse fra samme database på en gang.

FACTORY

CASE 5

SYSTEMKONFIGURATION

Et system skal bruge

- Database connection
- API keys
- Log-niveau
- Miljø (dev/test/prod)

Alle dele af systemet har brug for adgang til disse værdier.

PROBLEMET

- Hvis hver klasse selv loader konfigurationen -> spil og inkonsistens
- Hvis vi sender konfigurationen rundt overalt -> rodet
- Hvis flere instanser eksistere -> risiko for divergerende state

BEHOV

Vi ønsker:

- En fælles instans
- Kontrolleret adgang
- Global tilgængelighed
- Ens state

CASE 6

RAPPORTGENERATOR

Et system skal kunne generere rapporter i:

- PDF
- CSV
- JSON Brugeren vælger format via UI.

PROBLEMET

Hvis resten af systemet selv skal vælge:

- Hvem opretter objekterne?
- Hvor skal if/else ligge?
- Hvad sker, når vi tilføjer Excel?

Vi risikere:

- Tæt kobling
- Mange ændringer ved nye typer

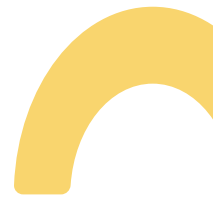
BEHOV

Vi ønsker:

- Et sted der håndtere oprettelse
- Resten af systemet skal være ligeglad med typen
- let at udvide

ASSIGNMENT

I KAN MED FORDEL ARBEJD MED TIDLIGERE OPGAVER.



NEXT

